XML Schema and Data Types

Modern XML Validation Using XSD and Type Definitions

Presented by : Manal Asghar

Introduction to XML Schema (XSD)

What is XML Schema?

- XML Schema is also called XSD (XML Schema Definition).
- It is a modern method used to validate XML documents.
- XSD defines the structure and rules of XML data.
- XML Schema controls which elements and attributes are allowed.
- It also defines data types for XML elements.
- XSD is more powerful and flexible than DTD.
- XML Schema is written in XML syntax.
- It is widely used in web applications and enterprise systems.

Main Purpose of XML Schema

- Validate XML documents
- Define XML structure
- Apply data type checking
- Improve data consistency
- Reduce invalid data errors

Why XML Schema is Important

Importance of XSD

- XML Schema ensures XML follows predefined rules.
- It supports advanced validation techniques.
- Developers can define accurate data types.
- XSD improves reliability of XML documents.
- It helps systems exchange data securely.
- XML Schema supports reusable components.
- Large systems use XSD for data validation.

Benefits of XML Schema

- Strong validation support
- Better than DTD
- Supports namespaces

- Supports complex data structures
- Easy maintenance and scalability

Difference Between DTD and XML Schema

DTD vs XSD

DTD	XML Schema (XSD)
Old validation method	Modern validation method
Not written in XML	Written in XML syntax
Limited data types	Supports many data types
Less flexible	More flexible
No namespace support	Supports namespaces
Weak validation	Strong validation

Summary

- DTD is simple but limited.
- XSD provides advanced validation features.
- XML Schema is preferred for professional systems.

Structure of XML Schema

Basic XSD Structure

- XML Schema starts with `<xs:schema>`.
- `xs` is the standard namespace prefix.
- Elements are declared using `<xs:element>`.
- Data types are assigned using XML Schema types.

Basic Example

```
<?xml version="1.0"?>

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

 <xs:element name="student" type="xs:string"/>

</xs:schema>
```

Explanation

- `xs:schema` defines schema document
- `xs:element` defines XML element
- `xs:string` defines text data type

Simple Types in XML Schema

Understanding Simple Types

- Simple types contain only text values.
- They cannot contain child elements.
- XML Schema provides many built-in simple data types.
- Simple types improve data validation.

Common Simple Data Types

- xs\:string
- xs\:int
- xs\:decimal
- xs\:boolean
- xs\:date
- xs\:float
- xs\:double

Example

```
<xs:element name="name" type="xs:string"/>
<xs:element name="age" type="xs:int"/>
<xs:element name="dob" type="xs:date"/>
```

Advantages

- Prevents invalid input
- Improves data quality
- Makes XML validation stronger

Complex Types in XML Schema

Understanding Complex Types

- Complex types contain multiple elements or attributes.
- They are used to create nested XML structures.
- Complex types help organize XML data logically.
- They are useful in real-world applications.

Example

```
<xs:complexType name="studentType">
 <xs:sequence>
  <xs:element name="name" type="xs:string"/>
  <xs:element name="age" type="xs:int"/>
 </xs:sequence>
</xs:complexType>
```

Explanation

- `complexType` creates structured data
- `sequence` defines element order
- Multiple child elements are allowed

XML Schema Data Types

Common Data Types Used in XSD

String Data Type

```
<xs:element name="city" type="xs:string"/>
```

Integer Data Type

```
<xs:element name="marks" type="xs:int"/>
```

Date Data Type

```
<xs:element name="birthDate" type="xs:date"/>
```

Boolean Data Type

```
<xs:element name="status" type="xs:boolean"/>
```

Pattern Validation

```
<xs:pattern value="[A-Z][a-z]+"/>
```

Benefits

- Accurate data validation
- Structured XML content
- Reduced data entry errors

Restriction and Facets

Restriction and Facets in XSD

- Restrictions limit the allowed values.
- Facets define validation conditions.
- They help control XML data accurately.
- Restrictions improve security and consistency.

Common Facets

- minLength
- maxLength
- pattern
- enumeration
- minInclusive
- maxInclusive

Example

```
<xs:simpleType name="ageType">
 <xs:restriction base="xs:int">
  <xs:minInclusive value="18"/>
  <xs:maxInclusive value="60"/>
 </xs:restriction>
</xs:simpleType>
```

Explanation

- Age must be between 18 and 60.
- Invalid values will fail validation.

Creating Nested Structures

Nested Structures with Complex Types

- Complex types support parent-child relationships.
- Nested structures represent real-world data.
- XML becomes more organized and readable.

- Large XML systems depend on nested structures.

Example

```
<student>
 <personalInfo>
  <name>Ali</name>
  <age>20</age>
 </personalInfo>
</student>
```

Advantages

- Better organization
- Easy data management
- Real-world structure representation
- Improved readability

Practical Task — Student Record XML

Create Student Record XML

student.xml

```
<?xml version="1.0"?>

<student>
 <name>Ali Khan</name>
 <age>21</age>
 <email>ali@gmail.com</email>
 <dob>2004-05-12</dob>
</student>
```

Purpose

- Store student information
- Validate XML using XSD
- Apply validation rules

Practical Task — Create XSD Schema

student.xsd

```
<?xml version="1.0"?>

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

 <xs:element name="student">
  <xs:complexType>
   <xs:sequence>

    <xs:element name="name" type="xs:string"/>
    <xs:element name="age" type="xs:int"/>
    <xs:element name="email" type="xs:string"/>
    <xs:element name="dob" type="xs:date"/>

   </xs:sequence>
  </xs:complexType>
 </xs:element>

</xs:schema>
```

Features Used

- Complex type
- Sequence
- String data type
- Integer data type
- Date data type

Practical Task — Add Validation Rules

Applying Restrictions

```
<xs:element name="age">
 <xs:simpleType>
  <xs:restriction base="xs:int">
   <xs:minInclusive value="18"/>
   <xs:maxInclusive value="30"/>
  </xs:restriction>
```

```
 </xs:simpleType>
</xs:element>
```

Email Pattern Validation

```
<xs:pattern value=".+@.+\\..+"/>
```

Validation Rules

- Age must be between 18 and 30
- Email must follow correct format
- Invalid data will generate errors

XML Validation with XSD

How to Validate XML Using XSD

Validation Steps

1. Create XML file
2. Create XSD schema
3. Link XML with XSD
4. Run validator
5. Check errors and validation results

XML Linking Example

```
<?xml version="1.0"?>

<student
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:noNamespaceSchemaLocation="student.xsd">

 <name>Ali Khan</name>
 <age>21</age>
 <email>ali@gmail.com</email>
 <dob>2004-05-12</dob>

</student>
```

Validation Tools

- VS Code XML Extension

- Online XML Validators
- Eclipse IDE
- NetBeans IDE

Advantages of XML Schema

Benefits of Using XSD

- Strong validation support
- Better than DTD
- Supports advanced data types
- Supports namespaces
- Improves XML accuracy
- Flexible and reusable
- Ideal for enterprise systems

Real World Applications

- Banking Systems
- Student Management Systems
- E-commerce Applications
- APIs and Web Services
- Enterprise Data Exchange

Conclusion

Summary

- XML Schema is a modern validation method.
- XSD defines XML structure and data types.
- Simple types store text values.
- Complex types create nested structures.
- Restrictions and facets improve validation.
- XML validation ensures reliable data.
- XSD is widely used in professional applications.